CS 362 In-Class Exercise 4: Project Beta Testing

Your name: Kanstantsin Nechyparenka

Project team number that you are testing (e.g., team 1): team 14

PART-1: Organization and Purpose (2 pts)

Q1) Does the repository provide a README explaining the purpose of the software? If yes, based on reading that documentation, do you understand all the interesting features provided by the software? Do you have any advice to improve that documentation?

Yes. The README.md explains the purpose of the software, more specifically that Chess++ is a terminal-based chess program that interfaces with the Stockfish engine, aimed at both beginners (learning legal moves / piece movement) and advanced players (analysis, evaluation, best lines).
I mostly understand all the interesting features. The README states the key capabilities well:

- Play full games vs Stockfish
- Analyze a position (depth-based evaluation)
- Show legal moves
- Set custom positions via FEN
- Engine info / verification
- Menu-driven terminal UI

My advices to improve this root-level README and overall documentation are:

- Add a short “What’s operational right now?” section with 1–2 concrete use cases. This is required by class and makes it easier to distinguish between operational and non-operational functionality.
- Clarify platform + Stockfish execution details. Exactly which OS you’ve tested on (Linux/macOS/Windows).
- Add a “How to test” section. You currently have empty readme in /tests, I would suggest to fill it out.
- Add a small “Repo map” section for more convenient repo navigation.

PART-2: Installation and Setup (1 pts)

Q2) Is the documentation to install or setup the software available? (Note that for a web application, it would be a URL to access the website and instructions to host the website on a server). When following the instructions, do you face any difficulties while installing the software (accessing the URL for a website)? If yes, please explicitly state what issues you encountered, so that the project team can fix them.
NOTE: If you are testing a web application, then you do not need to set up a web server and try hosting the web application. Just go through the documentation to find out if it clearly explains the steps to host the website.

Yes, the repository provides installation and setup instructions. The README explains the required software (C++ compiler, Make and the included Stockfish engine) and provides simple commands such as make run to compile and start the application. The instructions are straightforward and mostly easy to follow.
However, during testing I noticed an important environment dependency. The build instructions appear to work smoothly when using the OSU Flip servers, which already have the required compiler and development tools installed. When running the project outside of that environment (for example on a local machine), the instructions are not always sufficient, because the README does not explain how to install the required compiler or tools locally.
Consider adding this to your documentation to improve its quality:

- Specify the environment (@flip)
- Provide instructions on how to connect to servers
- Alternatively, include instructions for installing the required tools locally, such as:
    - Installing g++ and make
    - Ensuring the compiler supports C++11 or later
    - Verifying that the stockfish executable has the correct permissions on Unix systems

Overall, the build process itself works correctly once the environment is properly configured. Expanding the documentation with clearer setup instructions for both Flip servers and local machines would make the repository easier for external users and beta testers to run successfully.

PART-3: Functional and Non-Functional Testing (2 pts)

Q3) Select a use case for the application-under-test and use your creativity to test the application in different possible ways. For example, if you are testing a login functionality, then test the sign up feature, sign in, adding invalid credentials, special characters, etc. Please provide the details of the use case you tested on the software by describing exactly what all you did and in what order? Make sure you are making notes while doing this. If you find any issues (e.g., something that was confusing, incorrect, or not working at all), please provide as many details as you can to replicate the issues so that the team can fix them.

For testing the application, I focused on the core gameplay use case: starting a new game and performing different types of chess moves to verify move validation and board updates.

I built and ran the program using make run, then started a new game from the initial position. I first tested basic pawn moves such as e4, e5, and d4, which executed correctly and updated the board as expected. I then tested knight movement(e.g., Nf3, Nc6) to confirm correct L-shaped movement.

Next, I tested invalid move scenarios including attempting to move through pieces or moving the opponent's piece. In these cases, the program correctly returned an invalid move result, indicating proper move validation.

I also tested captures and special rules:

- Pawn capture (e4, d5, exd5)
- Castling
- En passant
- Pawn promotion (e.g., b8=Q)

All scenarios behaved as expected and matched the test results included in the repository.

The only bug I could find was in the main menu (first after running the pop up menu). If instead of listed numbers you will try to input ‘a’ or any other character the program breaks.

From a non-functional perspective, the engine responded quickly when calculating moves and analyzing positions. Communication with Stockfish worked correctly and helper commands were loading

One issue in the documentation is that the assignment requires a clearly defined operational use case for beta testing in the root-level README, but this section is currently missing. Adding it would make testing instructions clearer for users.

Another issue that I found while running the test is that Chess++ CI is either set up wrongly or all your commits don’t pass the integration test. I would recommend checking your CI logic.